

Fourier Language

Research Foundation & Architecture Principles

Key Papers for Implementation v0.1

Mario Michael Heinrich

Founder of easy-job.ai

github.com/Engineer1080

Januar 2026

1. Überblick

Dieses Dokument sammelt die wichtigsten wissenschaftlichen Papers und Forschungsergebnisse, die als Grundlage für die Architektur der Fourier-Programmiersprache dienen. Fourier ist darauf ausgelegt, GPU/TPU-beschleunigte, parallelisierte Berechnungen für Machine Learning und Grafikrendering zu ermöglichen. Die hier aufgeführten Papers decken die Kernbereiche ab:

- MLIR Compiler-Infrastruktur für heterogenes Computing
- Triton GPU-Programmierung und Kernel-Optimierung
- Work-Stealing Scheduler für parallele Ausführung
- Automatische Differenzierung für ML-Workloads
- Lineare und Affine Typsysteme für Speichersicherheit

2. MLIR: Compiler-Infrastruktur

MLIR (Multi-Level Intermediate Representation) ist die empfohlene Compiler-Infrastruktur für Fourier. MLIR ermöglicht die Darstellung von Code auf mehreren Abstraktionsebenen und ist speziell für heterogenes Computing (CPU/GPU/TPU) konzipiert.

2.1 Kernpaper zu MLIR

The MLIR Transform Dialect: Your Compiler Is More Powerful Than You Think

Lücke et al., CGO 2025

Zeigt wie MLIR Performance-Engineers feingranulare Kontrolle über Compiler-Optimierungen gibt. Der Transform-Dialekt ermöglicht es, bestehende Compiler-Features präzise zu kombinieren ohne neue Passes zu schreiben. Relevant für Fourier's Ansatz, dem Compiler intelligente Entscheidungen zu überlassen, aber Entwicklern explizite Kontrolle (`.gpu()`/`.cpu()`) zu geben.

MLIR: Scaling Compiler Infrastructure for Domain Specific Computation

Google Research, 2020

Das grundlegende MLIR-Paper. Erklärt wie MLIR Software-Fragmentierung adressiert, Compilation für heterogene Hardware verbessert und die Kosten für domänenspezifische Compiler reduziert. Zentral für Fouriers Multi-Level-Ansatz.

Enhancing Compiler Design for Machine Learning Workflows

Int. Journal of Science and Research Archive, 2025

Diskutiert MLIR's modularen Pass-Manager und wie er Optimierungen auf verschiedenen Abstraktionsebenen koordiniert. Besonders relevant für quantization passes, operator fusion und Hardware-spezifische Optimierungen in Fourier.

ARIES: An Agile MLIR-Based Compilation Flow for Reconfigurable Devices with AI Engines

FPGA 2025

Zeigt unified MLIR-basierte Darstellung für AIE-Architekturen. Erreicht bis zu 87% AIE-Effizienz bei Skalierung auf hunderte AIEs. Demonstriert MLIR's Fähigkeit, globale und lokale Optimierungen durchzuführen - wichtig für Fourier's GPU/TPU-Integration.

2.2 Architektur-Prinzipien für Fourier

Multi-Level IR: Fourier sollte verschiedene Abstraktionsebenen unterstützen - von High-Level ML-Operationen bis zu Low-Level GPU-Kernels.

Dialect-System: Eigene Fourier-Dialekte für ML, Grafik und Linear Algebra, die in MLIR's Ecosystem integriert werden.

Progressive Lowering: Schrittweise Transformation von High-Level zu Low-Level Darstellungen mit Optimierungen auf jeder Ebene.

Hardware-Abstraction: MLIR's Fähigkeit, für verschiedene Targets (CUDA, ROCm, TPU) zu kompilieren nutzen.

3. Triton: GPU Kernel Programming

Triton ist eine Python-ähnliche Sprache von OpenAI für GPU-Programmierung, die viele Low-Level-Details automatisiert. Triton zeigt, wie man eine produktive, High-Level-Syntax mit Hardware-Performance verbinden kann - eine Kernidee für Fourier.

3.1 Kernpaper zu Triton

Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations

Tillet et al., MAPL 2019

Das Original-Paper das Tritons Blocked Program Representation einführt. Automatisiert Memory Coalescing, Shared Memory Allocation und Instruction Scheduling. Zeigt wie man FP16 Matrix Multiplication mit cuBLAS-Performance in <25 Zeilen Code erreicht.

OpenAI Triton on NVIDIA Blackwell Boosts AI Performance

NVIDIA Technical Blog, April 2025

Aktuellste Entwicklungen in Triton für neueste GPU-Architekturen. Demonstriert wie Triton mit neuer Hardware Schritt hält. Wichtig für Fourier's Zukunftssicherheit.

Democratizing AI Accelerators and GPU Kernel Programming using Triton

Red Hat Emerging Technologies, November 2024

Erklärt Tritons Device-Independent Frontend und Device-Dependent Backend Compilation. Zeigt wie Vendor-spezifische Komplexität aus dem User-Code herausgehalten wird - ein Schlüsselprinzip für Fourier.

Triton kernel performance on RISC-V CPU

RISC-V International, November 2024

Demonstriert Triton-CPU mit MLIR's vector dialect für effektive Vektorisierung. Zeigt dass Tritons Ansatz auch für CPUs funktioniert - relevant für Fourier's .cpu() Modifier.

3.2 Architektur-Prinzipien für Fourier

Block-Level Parallelism: Statt SIMT (Single Instruction Multiple Thread) nutzt Triton Block-Operations. Fourier sollte ähnlich denken.

Automatische Optimierungen: Memory Coalescing, Shared Memory Management, Loop Fusion automatisch vom Compiler durchführen lassen.

Python-ähnliche Syntax: Produktivität durch vertraute Syntax, aber native Performance durch intelligente Compilation.

Hardware-Agnostic IR: Triton-IR als Zwischendarstellung, dann zu Hardware-spezifischem Code (PTX, AMDGPU, etc.) lowern.

4. Work-Stealing Scheduler

Work-Stealing ist ein bewährter Ansatz für dynamisches Load-Balancing in parallelen Systemen. Für Fourier's "parallelize by default" Philosophie ist ein effizienter Work-Stealing Scheduler essentiell.

4.1 Kernpaper zu Work-Stealing

Scheduling Multithreaded Computations by Work Stealing

Blumofe & Leiserson, JACM 1999

Das klassische Work-Stealing Paper. Beweist dass für fully-strict Computations die erwartete Zeit $T1/P + O(T_\infty)$ ist. Zeigt dass Space-Requirement $S1 \cdot P$ ist. Fundamentale Theorie für Fourier's Scheduler.

Proactive Work Stealing for Futures

Singer, Xu & Lee, PPOPP 2019

Erweitert Work-Stealing für Futures mit ProWS-Algorithmus. Relevant für Fourier's asynchrone GPU-Operationen und wenn `.gpu()` Calls nicht sofort blockieren sollen.

Work Assisting: Linking Task-Parallel Work Stealing with Data-Parallel Self Scheduling

de Wolff & Keller, ARRAY 2024

Novel Strategy für Mixing Data Parallelism (Loop Parallelism) mit Task Parallelism. Threads teilen ihre current data-parallel activity um anderen zu helfen. Sehr relevant für Fourier's verschachtelte Parallelisierung.

An Efficient Work-Stealing Scheduler for Task Dependency Graph

Lin et al., ICPADS 2020

Adaptive Strategy für die Anzahl der Working Threads basierend auf verfügbarem Task-Parallelismus. Erreicht 15% weniger Runtime mit 36% weniger Energy. Wichtig für Fourier's Effizienz.

4.2 Architektur-Prinzipien für Fourier

Provably Good Bounds: Implementierung sollte die theoretischen Bounds $T1/P + O(T_\infty)$ für Zeit und $S1 \cdot P$ für Space erreichen.

Adaptive Thread Count: Anzahl aktiver Threads dynamisch an verfügbaren Parallelismus anpassen um Energy zu sparen.

Nested Parallelism Support: Work-Stealing muss verschachtelte `.map()` Calls effizient handhaben ohne Thread-Explosion.

Low Overhead Deques: Private Deques mit minimalem Synchronisations-Overhead für lokale Operationen.

5. Automatische Differenzierung

Für ML-Workloads ist Automatic Differentiation (AD) essentiell. Fourier sollte AD als First-Class Feature einbauen, nicht als nachträgliche Library.

5.1 Kernpaper zu AD

MimlrADe: Automatic Differentiation in a Higher-Order Sea-of-Nodes IR

Ullrich, Hack & Leißa, CC 2025

Funktional inspirierte AD-Technik in higher-order, graph-based IR implementiert. Zeigt wie man funktionale AD-Ansätze mit Mutation und Taping kombiniert für Effizienz. Performance on par mit führenden Mutation/Taping-Techniken.

A Tensor Algebra Compiler for Sparse Differentiation

CGO 2024

Framework für efficient AD von sparse tensors. Wichtig weil Fourier oft mit sparse data (z.B. in Graphs) arbeiten wird.

Automatic Differentiation in ML: Where we are and where we should be going

NeurIPS 2018

Umfassender Survey über AD-Approaches. Diskutiert tape-based vs closure-based AD. Argumentiert für IR mit closures als first-class objects für einfachere AD-Implementation.

5.2 Architektur-Prinzipien für Fourier

Source-to-Source AD: AD sollte im Compiler integriert sein, nicht als separate Library. Ermöglicht bessere Optimierungen.

Mixed Mode AD: Sowohl Forward-Mode als auch Reverse-Mode unterstützen. Forward für wenige Inputs, Reverse für viele.

Sparse Tensor Support: Effiziente AD auch für sparse tensors um Memory und Compute zu sparen.

Integration mit Type System: Differentiable Functions als eigener Typ im Type System markieren.

6. Lineare und Affine Typen

Lineare und Affine Typsysteme ermöglichen Speichersicherheit ohne Garbage Collection und helfen Race Conditions zur Compile-Zeit zu verhindern. Für Fourier's parallele Ausführung und .address Attribute sind diese Konzepte zentral.

6.1 Kernpaper zu Linear/Affine Types

Affect: An Affine Type and Effect System

van Rooij & Krebbers, POPL 2025

Kombiniert Affine Types mit Effect System. One-shot effects deren Continuations at most once resumed werden. Wichtig für Fourier's GPU-Operations die nicht wiederholt werden sollten.

Functional Ownership through Fractional Uniqueness

Marshall & Orchard, OOPSLA 2024

Erweitert Ownership-Konzept mit fractional permissions. Ermöglicht Partial Borrowing von Datenstrukturen für besseren Parallelismus. Sehr relevant für Fourier's .address Konzept.

Linear Haskell: Practical linearity in a higher-order polymorphic language

Bernardy et al., POPL 2018

Zeigt wie man Linear Types non-invasiv in existierende Sprache integriert. Bestehende Datatypes können in linear parts verwendet werden. Gute Blaupause für Fourier.

Linear and Affine Types for Memory-Bounded Model Serving

Java Code Geeks, October 2025

Anwendung von Linear/Affine Types für ML Model Serving. Zeigt predictable deallocation ohne GC. Direktes Use-Case für Fourier's ML-Fokus.

6.2 Architektur-Prinzipien für Fourier

Affine Types für Speichersicherheit: Variablen können at most once bewegt werden. Verhindert use-after-free und double-free zur Compile-Zeit.

Ownership & Borrowing: Wie Rust: Klare Ownership-Rules, aber Borrowing für temporären Zugriff ohne Move.

.address als Read-Only: Das .address Attribut sollte read-only sein. Verhindert arbitrary memory access aber ermöglicht Introspection.

Zero-Cost Abstractions: Type-Checking zur Compile-Zeit ohne Runtime-Overhead. Keine GC nötig.

7. Implementierungs-Roadmap

Basierend auf den Papers ergibt sich folgende konkrete Architektur für Fourier:

Komponente	Technologie	Papers
Compiler Backend	LLVM + MLIR	Lücke 2025, Google MLIR
GPU Code Gen	Triton-inspiriert	Tillet 2019, NVIDIA 2025
CPU Scheduler	Work-Stealing	Blumofe 1999, Lin 2020
GPU Scheduler	Work-Stealing + Kernel Fusion	ARIES 2025
Auto-Diff	Source-to-Source	Ullrich 2025
Type System	Affine + Ownership	van Rooij 2025, Marshall 2024
Memory Mgmt	Linear Types	Bernardy 2018

7.1 Phase 1 Prioritäten

- **MLIR Integration:** Fourier-Dialect in MLIR erstellen mit custom operations für `.gpu()/cpu()/single()`
- **Basic Type System:** Affine Types für Ownership implementieren, `.address` Attribut
- **Simple Scheduler:** Work-Stealing für CPU mit adaptive thread count
- **GPU Backend:** MLIR → Triton-IR → CUDA/ROCm Code-Path aufbauen

8. Nächste Schritte

Um von der Spezifikation zur Implementation zu kommen, sollten folgende Schritte unternommen werden:

Paper Deep-Dive: Die 20+ Papers im Detail studieren, besonders die Implementation-Details der Algorithmen.

MLIR Tutorial: MLIR Tutorials durcharbeiten, eigene Dialekte erstellen lernen. MLIR Transform Dialect verstehen.

Triton Code Study: Triton's Source Code analysieren um zu verstehen wie Block-Level IR funktioniert.

Proof-of-Concept: Kleiner Interpreter in Python um Syntax zu testen und Design-Entscheidungen zu validieren.

LLVM Basics: LLVM IR generieren lernen, Simple Code-Generation testen.

Community Contact: Mit MLIR und Triton Communities in Kontakt treten, Feedback zu Fourier-Design einholen.

8.1 Hilfreiche Resources

- MLIR Documentation: <https://mlir.llvm.org/docs/>
- Triton GitHub: <https://github.com/triton-lang/triton>
- LLVM Getting Started: <https://llvm.org/docs/GettingStarted.html>
- Rust Book (für Ownership-Konzepte): <https://doc.rust-lang.org/book/>

• <https://github.com/ghc-proposals/ghc-proposals/blob/master/proposals/0111-linear-types.rst> Linear Haskell Proposal:

9. Fazit

Die gesammelten Papers zeigen, dass Fourier's Architektur auf solider wissenschaftlicher Grundlage steht. Die Kombination von MLIR für Compiler-Infrastruktur, Triton-ähnlichem GPU-Programming, Work-Stealing für Parallelismus, integrierter Auto-Diff und Affine Types für Speichersicherheit ist ein ambitioniertes aber machbares Ziel.

Die nächsten Monate sollten dem Studium dieser Papers und dem Aufbau eines Proof-of-Concept gewidmet sein. Mit einem funktionierenden Prototype können dann weitere Design-Entscheidungen empirisch getestet werden.

Fourier hat das Potenzial, eine neue Generation von ML- und Grafik-Entwicklern zu befähigen, hochperformanten Code zu schreiben ohne sich in Low-Level-Details zu verlieren. Die Forschung ist da - jetzt geht es an die Implementation.